

関西Ruby会議9 LT発表

しおまち / Shiomachi

学生がプログラミング言語処理系に向き合っ
て良かったこと！ Ruby編！



自己紹介

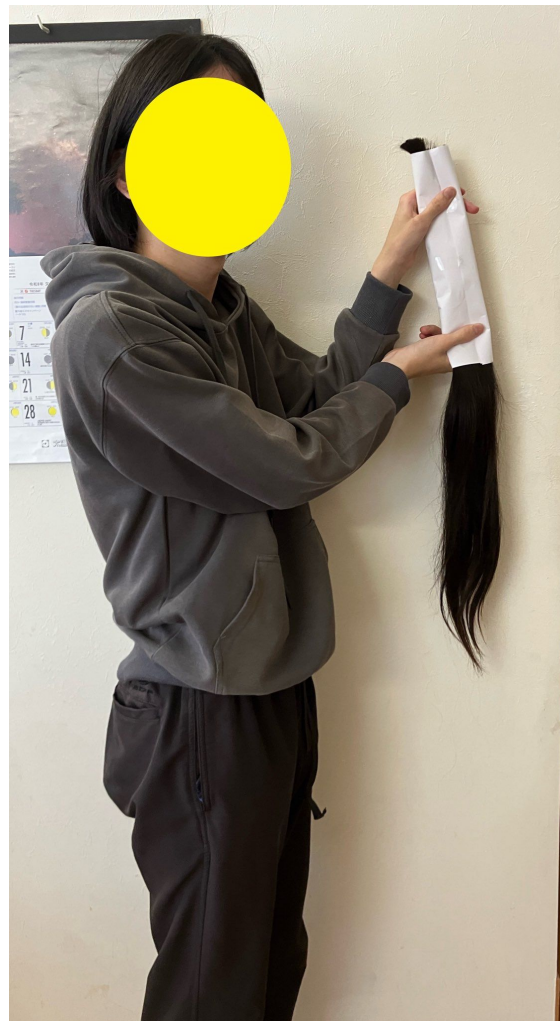
公開用なので省略



自己紹介

先週ヘアドネーションしました！
(医療用ウィッグのために
髪を伸ばして、寄付をすること)

レターパックで送付するというのがびっくり



自己紹介

The screenshot shows a GitHub repository for 'ShioPy0101'. The repository has 2 branches and 0 tags. The README file is selected, showing the following content:

Soft Loop Compiler Compiler (SLCC)

各プロジェクトでそれぞれ改良を重ねていたコンパイラコンパイラもどきを開発効率化のため集約しました。このリポジトリをサブリポジトリとして利用できます。ただし、パーサージェネレータの機能はないため、あくまでも"もどき"です。(故に原始プログラムの構文解析時に構文解析表を要する)

table.cpp : BNFから状態遷移表（構文解析表）を生成
analysis.cpp : 状態遷移表と原始プログラムを参照し構文木を生成

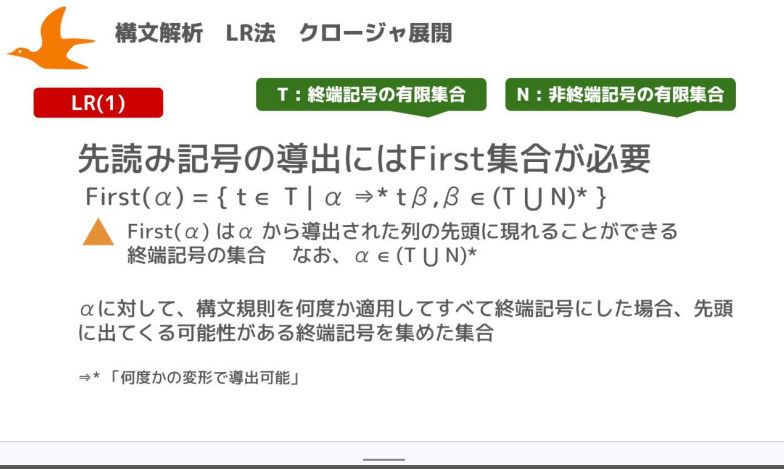
集約以前の開発記録（特に重要なものをpickup）

- 2023-01-19 開発及び学習を開始 (なんと、高校3年生である)
- 2023-09 LR(0)構文解析ができるようになりました
- 2023-10 LR(1)構文解析に改良
- 2024-11 構文木プログラムを改良
- 2025-01 字句解析ラベルによる解析ができるようになりました

The right sidebar shows repository statistics: No description, website, or topics provided. 41 commits, 5 stars, 0 watching, 0 forks, and 0 releases. The languages section shows C++ at 99.6% and Makefile at 0.4%.

拡張BNFから 構文解析表を生成するツールを自作（C++）

自己紹介



自己紹介

kyobashi.rbに参加したことがきっかけ！

> > Ruby 入門 1ヶ月目 < <

このカンファレンスに参加するために、
頑張ってキャッチアップしました！

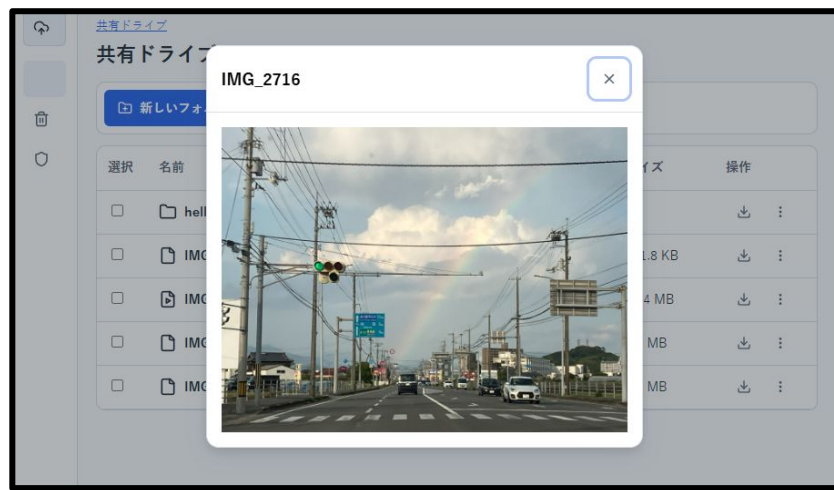
学生がプログラミング言語処理系を学んで よかったこと！ ～Ruby編～

- 1.実用的なこと
- 2.楽しいこと

1.実用的なところ

生成AI時代、静的解析を通じてより速く安全に開発できそう！

最近の開発



自宅サーバーにデプロイできるマルチテナントファイルサーバー

技術スタック

Ruby / **Rails API** / React / TypeScript / PostgreSQL
Nginx / Certbot / Ubuntu / Cloudflare DNS / Resend / Codex-CLI

所属コミュニティのために開発 . . .
大規模にする予定ではないが、
**自宅サーバーにデプロイするので
実装を強固にしたい！**

(port 443をポートフォワードイングする)

生成AIを活用しつつ楽したい！
-> そうだ、カスタムリントルールで担保しよう！

マルチテナントを守れてないコードを Lintで検出してみよう！

RuboCopを使って、ASTであそんでみよう！

たとえば：

```
es >  drive_itm.rb  
def set_active_drive_item  
  @drive_item = DriveItem.active.find_by(  
    id: params[:id]  
  )  
end
```

↑テナントで絞り込まずにIDで検索できると、
マルチテナントが危うくなる（意図しないものなら危険）

これを検出するLintツールを
つくってみよう

RuboCopでASTをみてみよう

```
def set_active_drive_item
  @drive_item = DriveItem.active.find_by(
    id: params[:id]
  )
end
```



ASTに変換

```
(def :set_active_drive_item
  (args)
  (ivasgn :@drive_item
    (send
      (send
        (const nil :DriveItem) :active) :find_by
        (hash
          (pair
            (sym :id)
            (send
              (send nil :params) :[]
              (sym :id)))))))
```

DriveItem

ファイルサーバーのファイルのこと

```
processed_source = RuboCop::AST::ProcessedSource.new(
  source,
  RUBY_VERSION.to_f,
  path
)

ast = processed_source.ast

puts ast
```

AST（抽象構文木）とは・・・
ソースコードを木構造にしたもの

たとえば：

きけん

```
def set_active_drive_item
  @drive_item = DriveItem.active.find_by(
    id: params[:id]
  )
end
```



ASTに変換

```
(def :set_active_drive_item
  (args)
  (ivasgn :@drive_item
    (send
      (send
        (const nil :DriveItem) :active) :find_by
      (hash
        (pair
          (sym :id)
          (send
            (send nil :params) :[])
            (sym :id)))))))
```

こうあってほしい

```
def set_active_drive_item
  current_user.organization.drive_items.active.find_by(
    id: params[:id]
  )
end
```



```
(def :set_active_drive_item
  (args)
  (send
    (send
      (send
        (send nil :current_user) :organization) :drive_items) :active) :find_by
    (hash
      (pair
        (sym :id)
        (send
          (send nil :params) :[])
          (sym :id))))))
```

Nodeをしぼる

```
ast.each_node(:send) do |node|  
  # DriveItemを直接検索している箇所を検出する  
  
  # node.method_nameがQUERY_METHODSに含まれているかどうかを確認  
  next unless QUERY_METHODS.include?(node.method_name)  
  
  # node.receiverがDriveItemを直接検索しているかどうかを確認  
  next unless drive_item_chain?(node.receiver)  
  
  puts "DriveItemを直接検索しています: #{node.source}"  
end
```

```
QUERY_METHODS = %i[  
  find  
  find_by  
  find_by!  
  where  
  exists?  
].freeze
```

```
# DriveItemを直接検索している箇所を検出する  
def drive_item_chain?(node)  
  return false unless node  
  
  if node.const_type?  
    return node.const_name == "DriveItem"  
  end  
  
  return false unless node.send_type?  
  
  drive_item_chain?(node.receiver)  
end
```

ast.each_nodeでASTのnodeを走査
これだけで DriveItem.active.find_by のようなコードを
検索できる

GitHub Actionsへの組み込み

```
# ASTを走査して、DriveItemを直接検索している箇所を検出する
# 違反を検出したnodeを格納しておく場所
violations = []

ast.each_node(:send) do |node|
  next unless QUERY_METHODS.include?(node.method_name)
  next unless drive_item_chain?(node.receiver)

  # DriveItemを直接検索している箇所を検出した場合、violationsに追加する
  violations << node
end

# violationsが空の場合は、正常終了する
if violations.empty?
  puts "#{path}: no violations"
  exit 0
end

# 検出場所の情報を出力する
violations.each do |node|
  location = node.location.expression

  puts "#{path}:#{location.line}:#{location.column + 1}: #{MESSAGE}"
  puts "  #{node.source}"
end

warn "#{violations.length} violation(s) detected"
exit 1
```

検出状況を表示するようにする

GitHub Actionsへの組み込み

```
tenant_lint:
  runs-on: ubuntu-latest

  steps:
    - name: Checkout code
      uses: actions/checkout@v6

    - name: Set up Ruby
      uses: ruby/setup-ruby@v1
      with:
        bundler-cache: true

    - name: Check tenant-scoped DriveItem queries
      shell: bash
      run: |
        mapfile -d '' files <<(
          find app/controllers app/services \
            -type f \
```

`.github/workflows/ci.yml`に記載

GitHub Actionsへの組み込み

```
def unscoped_drive_item_query_sample  
  DriveItem.find(params[:id])  
end
```

app/controllers/api/v1/admin/base_controller.rb:29:5: DriveItemをテナントスコープなしで直接検索しています
DriveItem.find(params[:id])

31 **Error:** Process completed with exit code 1.

テナントスコープのない検索を検出できるようになった！

ほかの言語のlinter : PHPStan

```
*
* @param array[] $xs
* @param string $key_column_name
* @return array<array> [キーA => [キーAを持つ$xsの要素, ...], キーB => [キーBを持つ$xsの要素, ...], ...]
*
* @template T of array
* @phpstan-param array<T> $xs
* @phpstan-return array<array<T>>
*/
± kana +1
public static function groupBy(array $xs, $key_column_name): array
{
    foreach ($xs as $x) {
        Util_Assert::isArray($x);
    }
    Util_Assert::nonEmptyString($key_column_name);

    $group_map = [];
    foreach ($xs as $x) {
        $key = $x[$key_column_name];
        if (isset($group_map[$key])) {
            $group_map[$key][] = $x;
        } else {
            $group_map[$key] = [$x];
        }
    }

    return $group_map;
}
```

ほかの言語のlinter : Go/Analysis

```
result := db.Table("orders").  
    Select("orders.id as order_  
    Joins("join products on ord  
    Where("orders.user_id = ?",  
    Scan(&orderHistory)
```

```
fmt.Printf("SelectorExpr: %s\n", pass.Fset.Position(sel.Pos()))  
  
if _, ok := terminalMethods[sel.Sel.Name]; !ok {  
    return true  
}  
  
methods := collectMethodChain(call)  
if len(methods) < 2 {  
    return true  
}  
  
fmt.Printf("Method chain: %s\n", strings.Join(methods, " -> "))
```

```
CallExpr: ./prob5.go:114:12  
SelectorExpr: ./prob5.go:114:12  
Method chain: Table -> Select -> Where -> Joins -> Scan  
CallExpr: ./prob5.go:114:12  
SelectorExpr: ./prob5.go:114:12
```

**抽象構文木の力を使って
オリジナルの カスタムリントルールを 作れる！**

生成AIと協調しよう！

大学の友達
WEBかゲーム開発している人が多い・・・

**2.パーサーに向き合う
楽しさってなんだろう！？**

私のプログラミング言語処理系への取り組み

高校卒業 ~ 学部1年のとき
BNFを手書きするところから！

私のプログラミング言語処理系への取り組み

パーサーに関する単語を何も知らない . . .

オートマトン

構文解析

クロージャ展開

私のプログラミング言語処理系への取り組み

どうやって木構造に？



token

変換



tree

私のプログラミング言語処理系への取り組み

Accept

構文解析が成功したことを表す

Reduce(構文規則)

- 1) その構文規則で還元 stackからその構文規則の長さだけpop
- 2) さらにstackのtopの値からgoto部を参照 その要素をstackにpush

Shift(遷移先の状態番号)

- 1) トークン列の現在のトークンと現在の状態番号から Action 表を参照
- 2) その表の要素をスタックにプッシュする

私のプログラミング言語処理系への取り組み

楽しさその1

頑張れば自分で実装できる！

この3つの操作をもとに
ただの1次元的なトークン列から
木構造が生成される！

私のプログラミング言語処理系への取り組み

楽しさその2

**プログラミング言語の
構文がなぜこうなってるのか、
考えることができる！**

私のプログラミング言語処理系への取り組み

楽しさその3

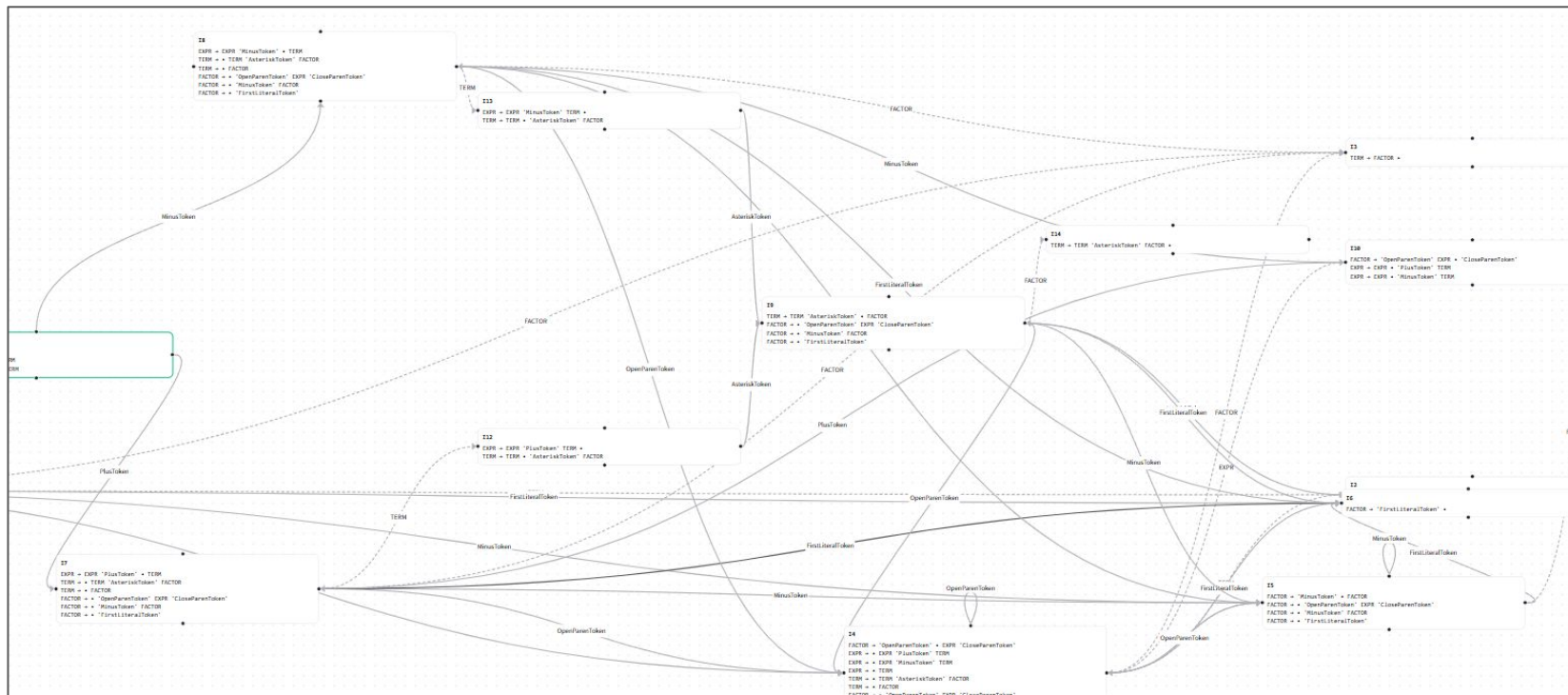
最大のおもしろさ . . .
自分で実装をする過程で
技術本に載ってるような単語をひとつひとつ
紐解いていくことができる！

オートマトン

構文解析

クロージャ展開

私のプログラミング言語処理系への取り組み

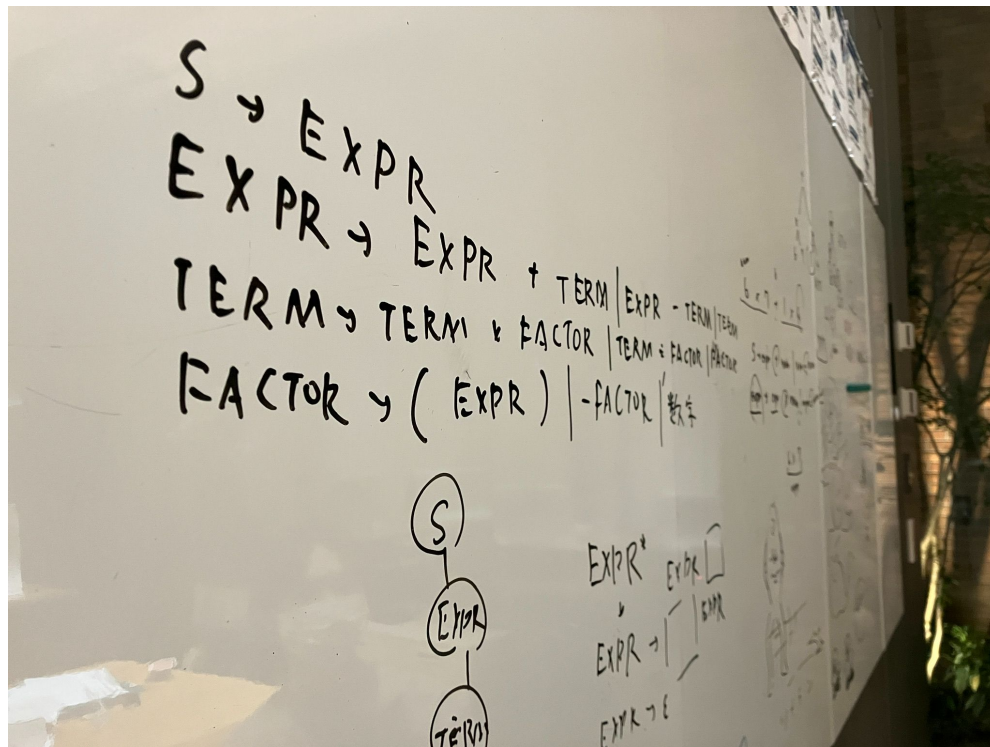


LRオートマトンを見るのが楽しくなります

まとめ

**プログラミング言語処理系を使えば
生成AI時代の開発が少し楽になるかも？**

まとめ



大学でも構文解析/構文木の力を広めたい！

ご清聴ありがとうございました

